

République Tunisienne

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université / École d'Ingénieurs

[Nom de l'établissement]

RAPPORT DE PROJET DE FIN D'ÉTUDES

En vue de l'obtention du Diplôme National d'Ingénieur

Spécialité : Sécurité des Systèmes Informatiques et des Réseaux

Automatisation avancée des tests d'intrusion par un système d'IA agentique : intégration de Metasploit & RAG et conception d'une console web d'opérations de sécurité (Strix Console)

Réalisé par : [Nom et Prénom de l'étudiant]

Encadrant académique : [Nom]

Encadrant professionnel : [Nom]

Année Universitaire 2025 – 2026

Dédicace

À mes chers parents, pour leur amour inconditionnel, leurs sacrifices et leur confiance indéfectible qui ont été ma plus grande source de force tout au long de ce parcours.

À ma famille et à mes proches, pour leur soutien constant et leurs encouragements.

À mes enseignants et encadrants, pour la qualité de leur accompagnement et la rigueur qu'ils m'ont transmise.

À mes amis et camarades, pour tous les moments partagés qui ont donné du sens à cette expérience.

Remerciements

Au terme de ce travail, je tiens à exprimer ma profonde gratitude à toutes les personnes qui ont contribué, de près ou de loin, à l'aboutissement de ce projet de fin d'études.

Je remercie chaleureusement mon encadrant académique pour sa disponibilité, ses conseils avisés et son suivi rigoureux, ainsi que mon encadrant professionnel pour la confiance qu'il m'a accordée et pour le cadre technique stimulant qu'il a su instaurer.

Mes remerciements s'adressent également aux membres du jury pour l'honneur qu'ils me font en acceptant d'évaluer ce travail, ainsi qu'à l'ensemble du corps enseignant pour la formation de qualité dont j'ai bénéficié.

Résumé

Ce projet de fin d'études porte sur l'automatisation avancée des tests d'intrusion à l'aide d'un système d'intelligence artificielle agentique fondé sur des grands modèles de langage (LLM). Le travail s'articule autour de deux contributions complémentaires. La première consiste à enrichir le moteur agentique open-source Strix en y intégrant le framework d'exploitation Metasploit et un service de connaissances RAG (Retrieval-Augmented Generation) : un outil unique et déterministe, `exploit_research`, interroge la base RAG puis recherche un module Metasploit et produit une recommandation « module d'abord », réduisant la charge de décision confiée au modèle. La seconde contribution est la conception et la réalisation d'une console web professionnelle d'opérations de sécurité, Strix Console, offrant l'authentification forte (cookies `httpOnly`, CSRF, MFA TOTP), un contrôle d'accès par rôles (RBAC), le pilotage et le suivi temps réel des campagnes (WebSocket), la visualisation de la sortie du moteur, une phase de post-exploitation, le triage des vulnérabilités, la comparaison de scans, la planification récurrente, l'export de rapports et un journal d'audit inaltérable. La plateforme, développée avec Next.js et FastAPI, a été validée par 39 tests automatisés et vérifiée de bout en bout.

Mots-clés :

tests d'intrusion, IA agentique, LLM, Metasploit, RAG, Strix, RBAC, MFA, FastAPI, Next.js, sécurité applicative.

Abstract

This graduation project addresses the advanced automation of penetration testing using an agentic AI system based on Large Language Models (LLMs). It delivers two complementary contributions. First, it augments the open-source Strix agentic engine with the Metasploit exploitation framework and a Retrieval-Augmented Generation (RAG) knowledge service: a single deterministic tool, `exploit_research`, queries the RAG, searches Metasploit and returns a module-first recommendation, offloading decision complexity from the model. Second, it designs and builds a professional security-operations web console, Strix Console, providing strong authentication (`httpOnly` cookies, CSRF, TOTP MFA), role-based access control, real-time assessment orchestration (WebSocket), engine-output visualisation, a post-exploitation phase, findings triage, scan diffing, recurring scheduling, report export and a tamper-evident audit trail. Built with Next.js and FastAPI, the platform is covered by 39 automated tests and verified end-to-end.

Keywords:

penetration testing, agentic AI, LLM, Metasploit, RAG, Strix, RBAC, MFA, FastAPI, Next.js.

Table des matières

Faites un clic droit ici → « Mettre à jour les champs » pour générer la table.

Liste des figures

- Figure 1.1 — Planning du projet (méthodologie itérative par phases)
- Figure 2.1 — Handoff d'exploitation : RAG + Metasploit (priorité au module)
- Figure 3.1 — Architecture globale de la plateforme Strix Console
- Figure 3.2 — Diagramme de cas d'utilisation
- Figure 3.3 — Diagramme de séquence : exécution d'un test
- Figure 3.4 — Matrice RBAC : permissions par rôle système
- Figure 4.1 — Pipeline d'évaluation en 5 phases
- Figure 5.1 — Page de connexion sécurisée (MFA)
- Figure 5.2 — Tableau de bord
- Figure 5.3 — Détail d'une campagne : pipeline, post-exploitation et triage
- Figure 5.4 — Écran de sortie du moteur (arbre d'agents, télémétrie, console)
- Figure 5.5 — Planification de tests récurrents
- Figure 5.6 — Gestion des rôles et permissions (RBAC)
- Figure 5.7 — Couverture de tests par module
- Figure 5.8 — Répartition des vulnérabilités par sévérité
- Figure 5.9 — Analyse comparative : approche manuelle vs plateforme

Liste des tableaux

- Tableau 1 — Besoins fonctionnels principaux
- Tableau 2 — Besoins non fonctionnels
- Tableau 3 — Pile technologique du projet
- Tableau 4 — Modèle de données (principales tables)
- Tableau 5 — Synthèse des résultats de validation

Liste des abréviations

Abréviation	Signification
LLM	Large Language Model (grand modèle de langage)
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control (contrôle d'accès par rôles)
MFA / TOTP	Multi-Factor Authentication / Time-based One-Time Password

CSRF	Cross-Site Request Forgery
API	Application Programming Interface
WS	WebSocket
CVE	Common Vulnerabilities and Exposures
ORM	Object-Relational Mapping
CI/CD	Continuous Integration / Continuous Deployment

Introduction générale

La transformation numérique des organisations s'accompagne d'une surface d'attaque en expansion permanente. Les tests d'intrusion (penetration testing) constituent l'un des moyens les plus efficaces pour évaluer, de façon offensive et contrôlée, la robustesse d'un système d'information. Toutefois, réalisés manuellement, ils demeurent coûteux en temps, difficilement reproductibles et fortement dépendants de l'expertise de l'opérateur.

L'émergence des grands modèles de langage (LLM) et des architectures dites « agentiques » ouvre la voie à une automatisation intelligente de ce processus : des agents autonomes planifient, exécutent et enchaînent des actions offensives en s'appuyant sur des outils réels. Le moteur open-source Strix illustre cette approche. Néanmoins, deux limites subsistent : d'une part, la phase d'exploitation reste fragile lorsqu'elle est entièrement confiée au raisonnement du modèle ; d'autre part, l'usage en équipe manque d'une interface d'opérations sécurisée offrant gouvernance, traçabilité et pilotage.

Ce projet répond à ces deux limites. Il intègre au moteur Strix le framework Metasploit et un service de connaissances RAG au travers d'un outil déterministe unique, puis conçoit et réalise une console web professionnelle, Strix Console, dédiée à l'orchestration et au suivi des campagnes de sécurité.

Le présent rapport est organisé en cinq chapitres. Le premier présente le cadre général du projet et la méthodologie adoptée. Le deuxième dresse un état de l'art des tests d'intrusion et de leur automatisation par IA. Le troisième détaille la conception et l'architecture. Le quatrième décrit les technologies et la réalisation. Le cinquième expose les tests et l'évaluation. Une conclusion générale synthétise les apports et ouvre des perspectives.

Chapitre 1 — Cadre général du projet

1.1 Introduction

Ce chapitre situe le projet dans son contexte, formule la problématique, présente la solution proposée et décrit la méthodologie de gestion adoptée pour mener le développement.

1.2 Contexte et problématique

Le pentest manuel souffre de plusieurs limites structurelles : durée d'exécution élevée, dépendance à l'expertise humaine, faible reproductibilité et difficulté de traçabilité. Les plateformes d'automatisation par IA agentique, comme Strix, apportent une réponse prometteuse mais présentent, en l'état, deux insuffisances :

- **Exploitation fragile** — en confiant l'intégralité de la décision d'exploitation au LLM, l'agent peut échouer à sélectionner ou paramétrer correctement un exploit, faute d'accès structuré à une base de connaissances et à un arsenal éprouvé.
- **Absence d'interface d'opérations** — l'outil est piloté en ligne de commande, sans gestion des utilisateurs, sans contrôle d'accès, sans suivi temps réel ni gouvernance, ce qui limite l'usage collaboratif et la valeur opérationnelle en entreprise.

La problématique retenue peut ainsi se formuler : comment fiabiliser la phase d'exploitation d'un agent de pentest fondé sur un LLM, et comment en faire un service d'opérations de sécurité sûr, gouverné et exploitable en équipe ?

1.3 Solution proposée

La solution se décline en deux volets complémentaires :

- **Volet 1 — Fiabilisation de l'exploitation.** Intégration de Metasploit et d'un service RAG dans le moteur Strix via un outil unique et déterministe, `exploit_research`, qui interroge le RAG, recherche un module Metasploit et renvoie une recommandation « module d'abord ». La logique de décision est déplacée du modèle vers du code Python vérifiable.
- **Volet 2 — Console d'opérations.** Conception d'une application web, Strix Console, offrant authentification forte, RBAC, pilotage et suivi temps réel des campagnes, visualisation de la sortie du moteur, post-exploitation, triage, comparaison de scans, planification récurrente, reporting et audit inaltérable.

1.4 Méthodologie de gestion de projet

Le développement a suivi une démarche itérative et incrémentale, organisée en phases successives livrant chacune un incrément fonctionnel testé et vérifié. Cette approche, proche de Scrum/Kanban, a

permis de sécuriser les fondations avant d'empiler les fonctionnalités, tout en conservant une application exécutable à chaque étape.

- **Intégration Metasploit + RAG** — outil exploit_research, playbooks et tests.
- **Phase 0 (Fondations)** — couche de données asynchrone, migrations, files et harnais de tests.
- **Phase 1 (Authentification)** — cookies httpOnly, CSRF, MFA TOTP, RBAC, sessions, audit.
- **Phase 2 (Temps réel)** — WebSocket, réconciliation des exécutions orphelines.
- **Phase 3 (Sortie moteur)** — arbre d'agents, console live, télémétrie (jetons, coût).
- **Phase 4 (Post-exploitation)** — 5^e phase du pipeline et événements typés.
- **Phase 5 (Domaine)** — triage, comparaison de scans, alertes, planification récurrente.
- **Phase 6 (Qualité)** — palette de commandes $\mathbb{K}$, observabilité, durcissement.

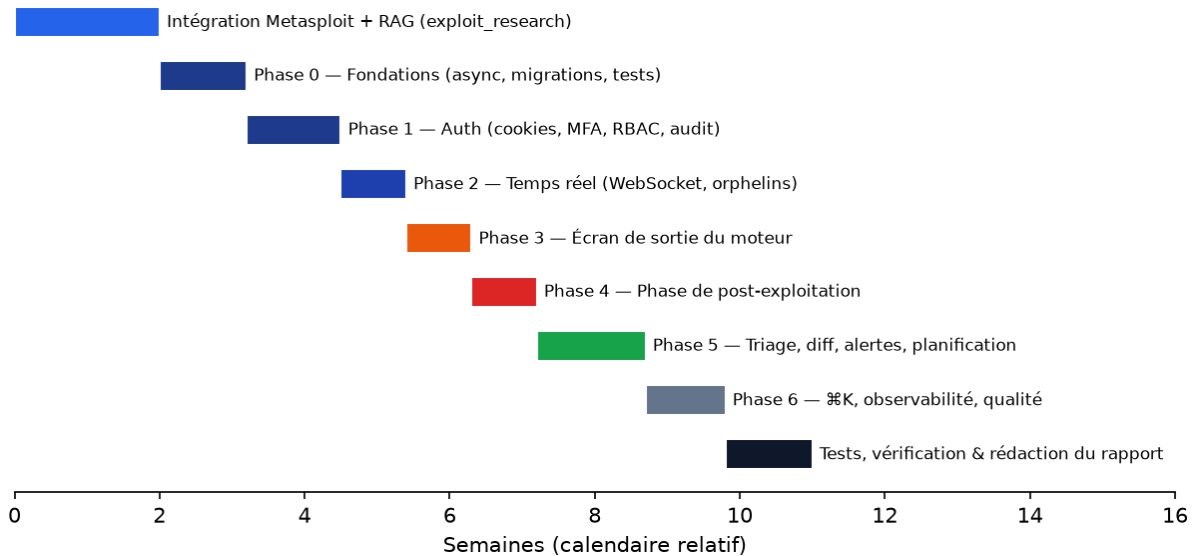


Figure 1.1 — Planning du projet (méthodologie itérative par phases).

1.5 Conclusion

Ce chapitre a défini le cadre, la problématique et la trajectoire du projet. Le chapitre suivant établit l'état de l'art sur lequel s'appuient les choix de conception.

Chapitre 2 — État de l'art

2.1 Introduction

Ce chapitre présente les fondements des tests d'intrusion, les approches d'automatisation par IA agentique et les briques techniques mobilisées (Strix, Metasploit, RAG), afin de positionner la contribution du projet.

2.2 Tests d'intrusion et Red Teaming

Un test d'intrusion reproduit, de manière autorisée et méthodique, le comportement d'un attaquant afin d'identifier et de démontrer des vulnérabilités exploitables. Le cycle de vie standard comprend la reconnaissance, l'évaluation des vulnérabilités, l'exploitation, la post-exploitation puis la rédaction du rapport. Le Red Teaming étend cette démarche à une simulation d'adversaire plus réaliste et furtive.

2.3 Automatisation par IA agentique et LLM

Un système « agentique » orchestre un ou plusieurs agents fondés sur un LLM capables de raisonner, de décomposer une tâche, d'appeler des outils et d'itérer à partir des retours d'exécution. Appliqué au pentest, ce paradigme automatise la planification et l'enchaînement des actions offensives. Des travaux comme PentestGPT ou AutoGPT ont démontré la faisabilité de cette approche, tout en révélant ses limites : hallucinations, décisions d'exploitation peu fiables et besoin d'un ancrage dans des connaissances vérifiées.

2.4 Briques mobilisées

- **Strix** — moteur multi-agent open-source de pentest ; un CLI côté hôte orchestre des agents qui exécutent des outils dans un bac à sable Docker (Kali).
- **Metasploit Framework** — arsenal d'exploitation de référence ; ses modules apportent vérification non destructive (check), gestion des cas limites et sessions structurées.
- **RAG (Retrieval-Augmented Generation)** — service qui indexe des connaissances d'exploitation et fournit, sur requête, un contexte et un guidage ancrés dans des documents réels plutôt que dans la seule mémoire du modèle.

La combinaison de ces briques permet d'ancrer le raisonnement de l'agent (RAG) tout en privilégiant un exécuteur éprouvé (Metasploit) lorsqu'un module existe.

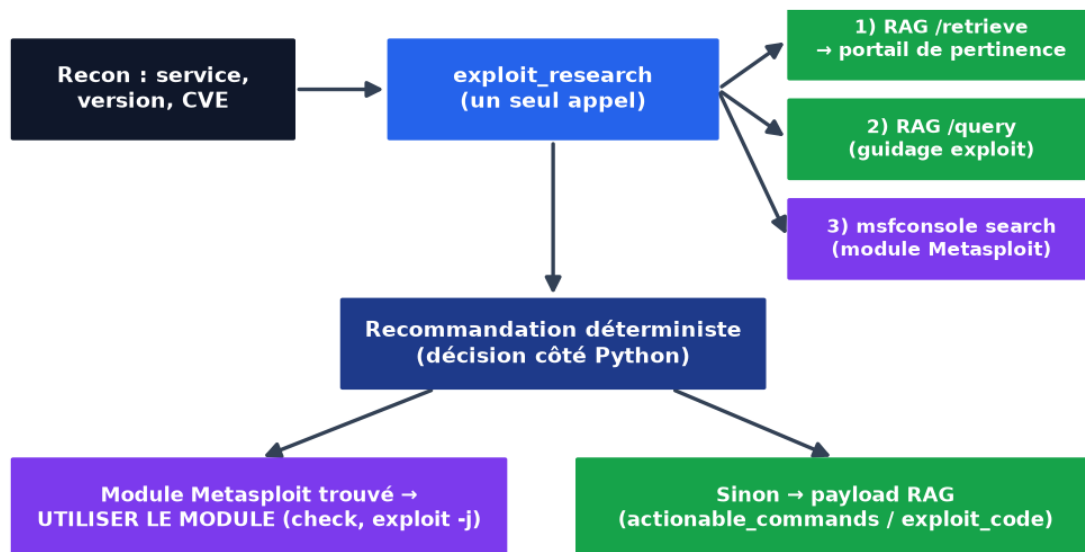


Figure 2.1 — Handoff d'exploitation : RAG puis Metasploit, avec priorité au module.

2.5 Positionnement du projet

Par rapport à l'existant, la contribution est double : (i) rendre l'exploitation déterministe en collapsant le protocole RAG + Metasploit dans un outil unique dont la décision est codée en Python ; (ii) doter la plateforme d'une console d'opérations sécurisée, absente des outils comparables, apportant gouvernance, temps réel et traçabilité.

2.6 Conclusion

L'état de l'art confirme la pertinence d'une approche hybride « IA agentique + arsenal éprouvé » et l'intérêt d'une interface d'opérations. Le chapitre suivant en présente la conception.

Chapitre 3 — Conception & Architecture

3.1 Introduction

Ce chapitre décrit l'architecture globale, les besoins fonctionnels et non fonctionnels, les diagrammes UML, le modèle de données et la conception de la sécurité.

3.2 Architecture globale

La plateforme adopte une architecture trois tiers. Le client (Next.js) communique en même origine avec un serveur Next.js qui relaie les appels `/api/*` vers un backend FastAPI asynchrone. Ce dernier expose les services métier, persiste les données dans PostgreSQL (SQLite en développement) et pilote le moteur Strix, lequel s'appuie sur Metasploit et le service RAG. Redis fournit files et cache.

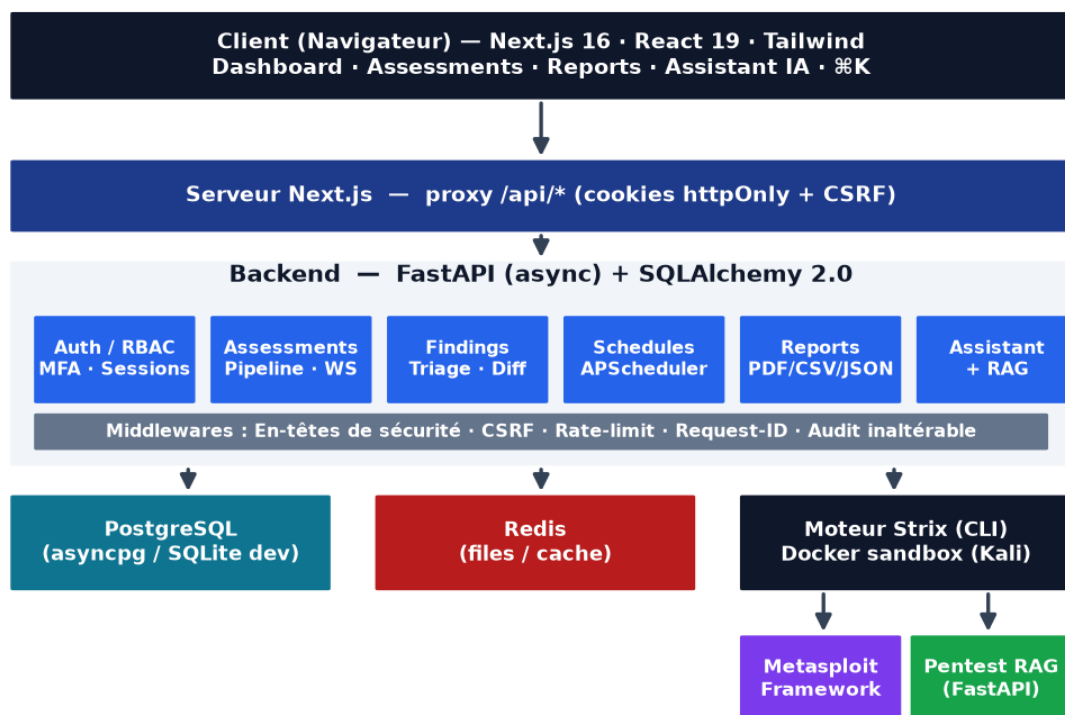


Figure 3.1 — Architecture globale de la plateforme Strix Console.

3.3 Besoins

3.3.1 Acteurs

Quatre rôles système sont définis, du moins au plus privilégié : Observateur (lecture seule), Analyste (création et exécution de tests, triage, export), Manager (supervision de toutes les campagnes, audit, lecture des utilisateurs) et Administrateur (accès complet).

3.3.2 Besoins fonctionnels

Réf.	Besoin fonctionnel
BF1	S'authentifier avec cookies sécurisés et second facteur (MFA TOTP)
BF2	Gérer les utilisateurs, les rôles et les permissions (RBAC)
BF3	Créer, lancer, annuler et supprimer un test d'intrusion
BF4	Suivre en temps réel le pipeline (état, phases, journal)
BF5	Visualiser la sortie du moteur (arbre d'agents, télémétrie, console)
BF6	Trier les vulnérabilités (statut, gravité, assignation)
BF7	Comparer deux scans d'une même cible (nouveaux / corrigés)
BF8	Planifier des tests récurrents (cron)
BF9	Exporter des rapports (PDF, CSV, JSON)
BF10	Consulter l'assistant IA et le journal d'audit

Tableau 1 — Besoins fonctionnels principaux.

3.3.3 Besoins non fonctionnels

Réf.	Besoin non fonctionnel
BNF1	Sécurité : jetons httpOnly, protection CSRF, MFA, en-têtes de sécurité
BNF2	Performance : backend asynchrone, temps réel via WebSocket
BNF3	Traçabilité : journal d'audit inaltérable (chaîné par hachage)
BNF4	Fiabilité : réconciliation des exécutions, tests automatisés
BNF5	Maintenabilité : migrations versionnées, séparation claire des couches
BNF6	Observabilité : identifiants de requête, sondes /readyz et /livez

Tableau 2 — Besoins non fonctionnels.

3.4 Diagrammes UML

3.4.1 Diagramme de cas d'utilisation

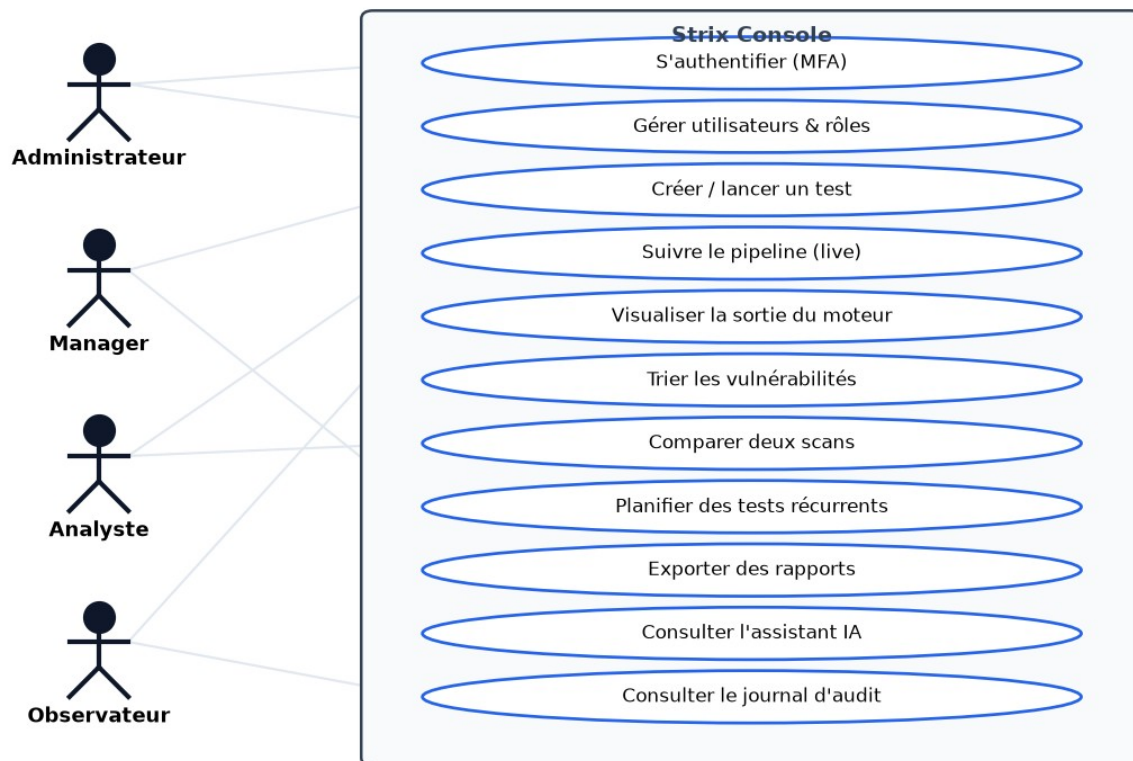


Figure 3.2 — Diagramme de cas d'utilisation de la plateforme.

3.4.2 Diagramme de séquence — exécution d'un test

Le scénario nominal d'exécution d'un test illustre l'enchaînement entre l'utilisateur, le frontend, l'API, le moteur et la base de données, ainsi que la diffusion des événements temps réel via WebSocket.

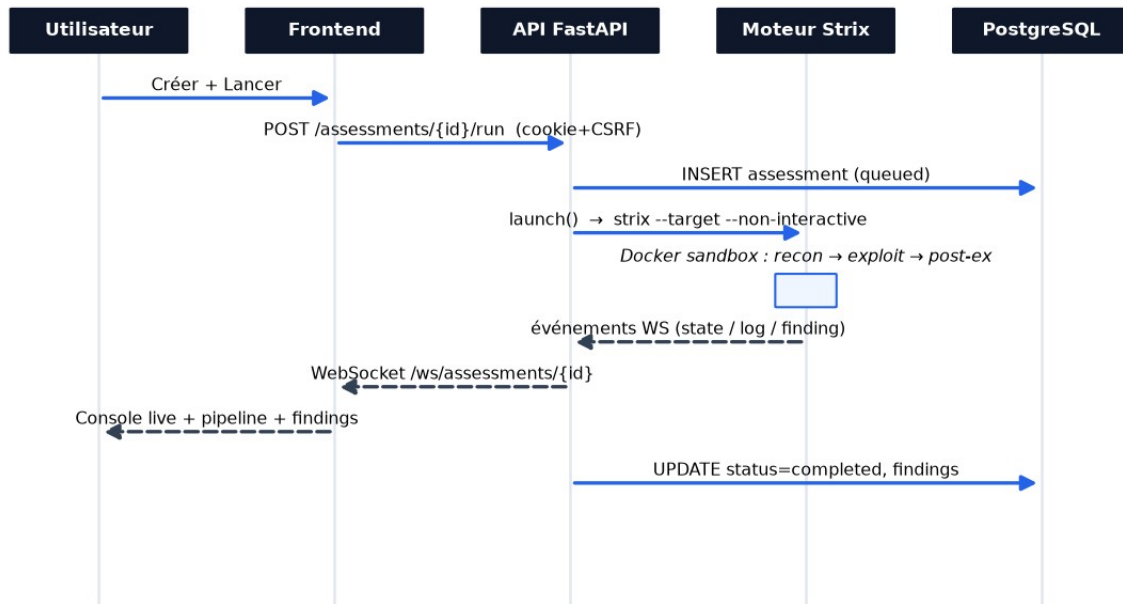


Figure 3.3 — Diagramme de séquence : exécution d'un test.

3.5 Modèle de données

Le modèle relationnel s'organise autour des entités suivantes, gérées via l'ORM SQLAlchemy 2.0 et versionnées par Alembic.

Table	Rôle
users / roles / role_permissions	Comptes, rôles et permissions (RBAC)
user_sessions	Sessions serveur (rotation, détection de réutilisation)
assessments / findings	Campagnes et vulnérabilités découvertes
post_ex_events	Événements de post-exploitation (session, privesc, exfil...)
schedules	Définitions de tests récurrents (cron)
log_entries / audit_events	Journaux d'exécution et journal d'audit chaîné
reports / chat_sessions / chat_messages	Rapports et conversations de l'assistant

Tableau 3 — Modèle de données (principales tables).

3.6 Conception de la sécurité

La sécurité repose sur une authentification par cookies httpOnly (jeton d'accès non lisible en JavaScript), une protection CSRF par double soumission, un second facteur TOTP obligatoire pour les rôles à

privilèges, une rotation des jetons de rafraîchissement avec détection de réutilisation, ainsi qu'un contrôle d'accès par rôles appliqué à chaque endpoint.

The image shows a matrix where rows represent roles and columns represent endpoints. Blue cells with a checkmark indicate granted permissions. The matrix is structured as follows:

Role	stats:read	assessment:read	assessment:create/run	finding:triage	schedule:manage	report:export	assessment:read:all	user:read	audit:read	user:manage	role:manage	settings:manage
Observateur	✓	✓										
Analyste	✓	✓	✓	✓	✓	✓						
Manager	✓	✓	✓	✓	✓	✓	✓	✓	✓			
Admin	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

Figure 3.4 — Matrice RBAC : permissions accordées par rôle système.

3.7 Conclusion

La conception établit une base sûre et modulaire. Le chapitre suivant décrit les technologies retenues et la réalisation.

Chapitre 4 — Technologies & Réalisation

4.1 Introduction

Ce chapitre présente la pile technologique, l'intégration Metasploit + RAG et la réalisation du backend et du frontend.

4.2 Pile technologique

Couche	Technologies
Frontend	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS, Recharts
Backend	FastAPI (async), SQLAlchemy 2.0, Pydantic v2, Uvicorn
Sécurité	Argon2, PyJWT, pyotp (TOTP), slowapi (rate-limit)
Données	PostgreSQL (asyncpg) / SQLite (aiosqlite), Alembic, Redis
Planification	APScheduler, croniter
Moteur	Strix (CLI multi-agent), Docker, Kali Linux, Metasploit, RAG
Qualité	pytest, ruff, GitHub Actions (CI)

Tableau 4 — Pile technologique du projet.

4.3 Intégration Metasploit & RAG (exploit_research)

L'intégration transforme un protocole multi-étapes en un unique appel d'outil déterministe. Dès que la reconnaissance fournit un service, une version ou une CVE, l'agent invoque exploit_research, qui : (1) interroge le RAG (/retrieve puis, après un portail de pertinence, /query) en contournant le proxy et avec un délai adapté ; (2) recherche un module via msfconsole ; (3) détecte l'adresse LHOST du bac à sable ; (4) produit une recommandation déterministe privilégiant le module Metasploit lorsqu'il existe, et retombant sur le payload RAG sinon. La logique de décision est ainsi codée et testée, et non laissée au modèle.

Extrait — construction de la commande du moteur avec instruction optionnelle :

```
cmd = ["strix", "--target", target, "--non-interactive",
      "--scan-mode", mode]
if instruction:
    cmd += ["--instruction", instruction]
```

4.4 Réalisation du backend

- **Authentification & RBAC** — connexion par cookies, MFA TOTP, sessions serveur avec rotation et détection de réutilisation, permissions resource:action vérifiées par dépendance.
- **Pipeline d'évaluation** — création, lancement, annulation ; exécution en tâche de fond et diffusion d'événements d'état, de journaux et de vulnérabilités.
- **Temps réel** — bus d'événements interne pontant le runner (thread) vers les abonnés WebSocket ; réconciliation au démarrage des exécutions orphelines.
- **Triage & comparaison** — mise à jour du statut, de la gravité et de l'assignation d'une vulnérabilité ; comparaison des vulnérabilités entre deux scans d'une même cible.
- **Planification** — APScheduler déclenche des tests récurrents selon une expression cron ; calcul de la prochaine occurrence via croniter.
- **Observabilité** — middleware d'identifiant de requête, sondes /readyz (base + moteur) et /livez ; journal d'audit inaltérable chaîné par hachage.

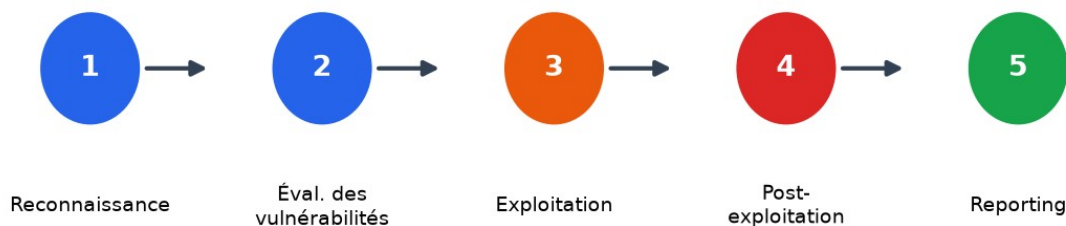


Figure 4.1 — Pipeline d'évaluation en 5 phases (dont la post-exploitation).

4.5 Réalisation du frontend

Le frontend, en Next.js (App Router), propose des modules dédiés : tableau de bord (statistiques et graphiques), campagnes et vue de détail (pipeline live, console, post-exploitation, triage, comparaison), rapports, journaux, audit, utilisateurs, rôles, planification, assistant IA et paramètres. Une palette de commandes (⌘K) offre une navigation rapide filtrée par permissions. L'interface est responsive, thématizable (clair/sombre) et cohérente grâce à un système de composants réutilisables.

4.6 Conclusion

La réalisation couvre l'ensemble des besoins définis, du moteur enrichi à la console d'opérations. Le chapitre suivant en évalue la qualité.

Chapitre 5 — Tests & Évaluation

5.1 Introduction

Ce chapitre présente l'environnement de test, la validation fonctionnelle, la couverture de tests et une analyse comparative avec l'approche manuelle.

5.2 Environnement de test

Les essais ont été menés sur un poste Windows 11 avec Docker Desktop, l'intégration ayant été validée sur une image de bac à sable Kali contenant Metasploit, contre une cible vulnérable de type application Drupal. Le backend a été exécuté en mode asynchrone (SQLite en développement, PostgreSQL en cible), le frontend via le serveur de développement Next.js.

5.3 Validation fonctionnelle

Chaque incrément a fait l'objet d'une vérification de bout en bout : connexion par cookies et MFA, application effective du RBAC (retour 403 pour un observateur sur une action d'administration), exécution et suivi temps réel d'une campagne, visualisation de la sortie du moteur avec télémétrie réelle (par exemple 6 requêtes LLM et 68 542 jetons sur une exécution), triage d'une vulnérabilité, comparaison de scans, planification récurrente et export de rapports. Côté intégration moteur, exploit_research a produit, sur une cible Drupal, une recommandation ancrée (CVE-2018-7600, Drupalgeddon2) avec sélection du module Metasploit.

Les captures d'écran ci-dessous illustrent les principales vues de l'interface réalisée.

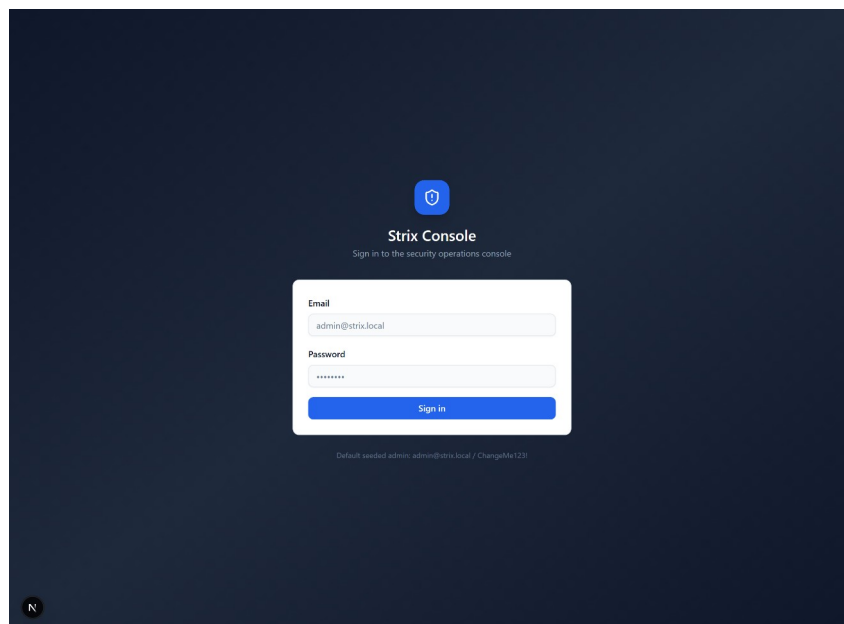


Figure 5.1 — Page de connexion sécurisée (cookies httpOnly, étape MFA).

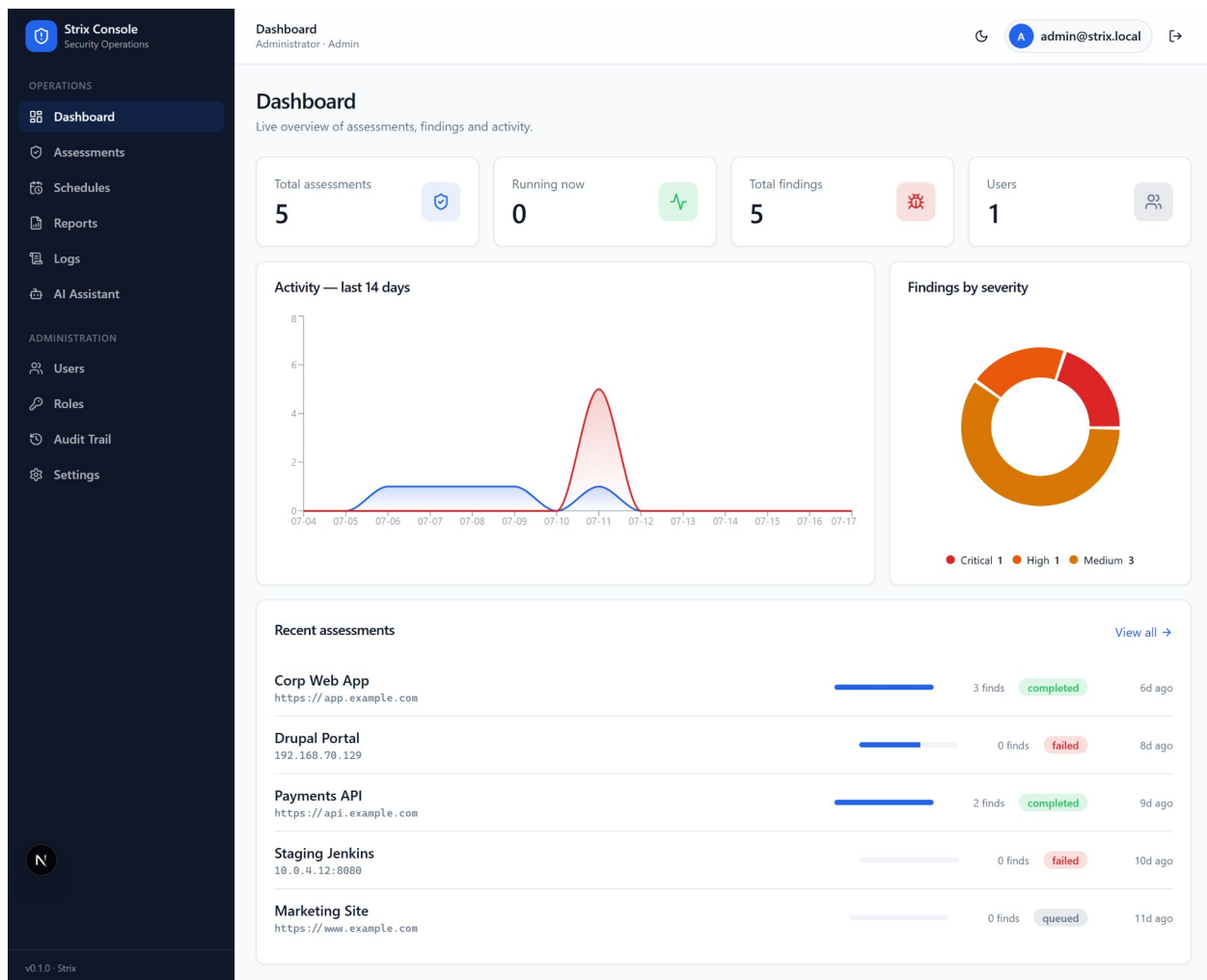


Figure 5.2 — Tableau de bord : statistiques, activité et répartition des vulnérabilités.

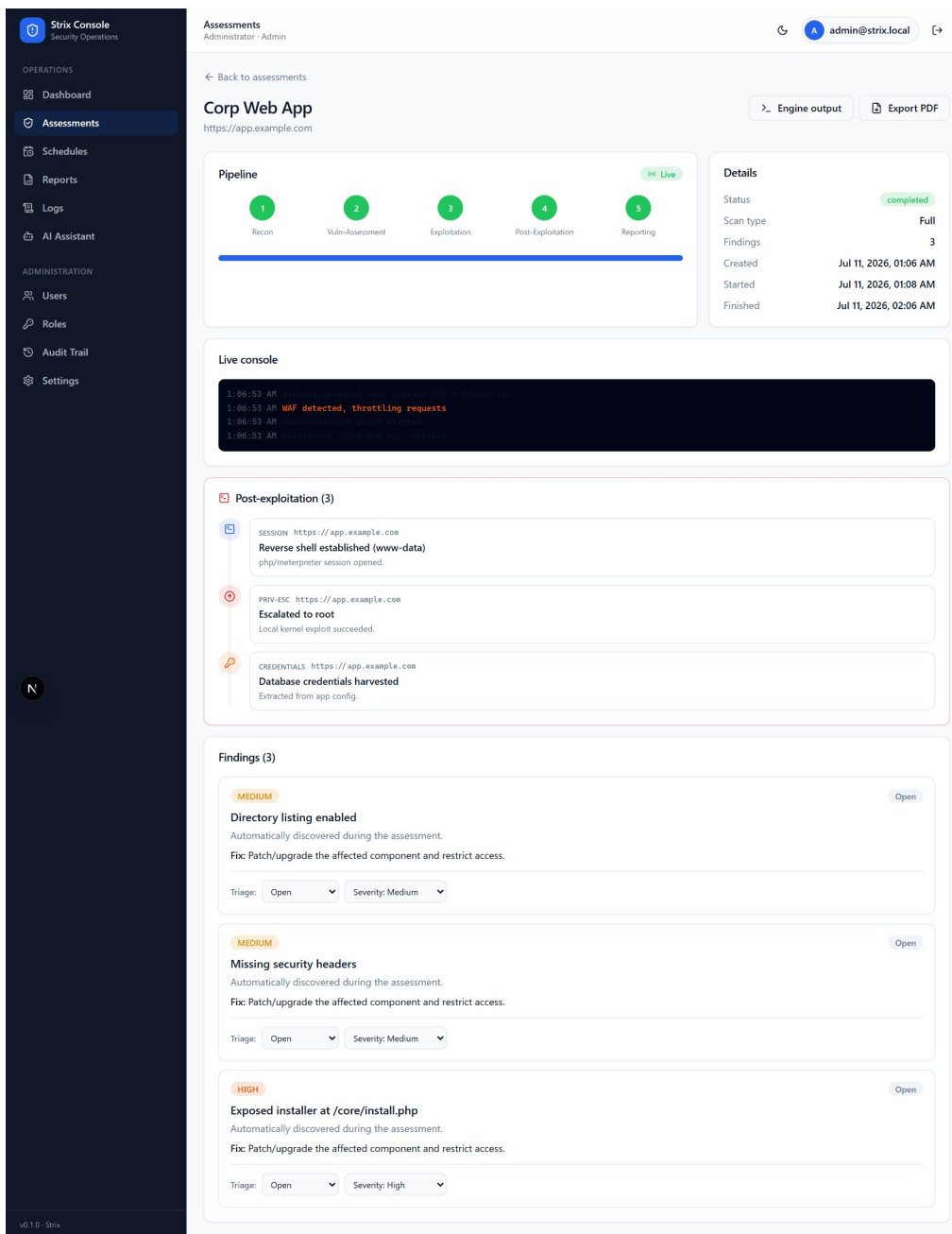


Figure 5.3 — Détail d'une campagne : pipeline en 5 phases, post-exploitation et triage des vulnérabilités.

Strix Console
Security Operations

OPERATIONS

Dashboard

Assessments

Schedules

Reports

Logs

AI Assistant

ADMINISTRATION

Users

Roles

Audit Trail

Settings

N
v0.1.0 - Strix

Assessments
Administrator · Admin

⌂ admin@strix.local ↗

← Back to assessment

Engine output

Live agent tree, token usage and streaming console from the Strix engine.

Live

LLM requests

6

Total tokens

68,542

68,483 in · 59 out

Est. cost

\$0

Elapsed

4m 4s

Agent tree

● strix FAILED

IP Addresses: - host.docker.internal Exploit Intelligence: - After reconnaissance identifies services and versions, use the...

Console

Q Filter output...

All ▾

1:06:53 AM INFO exploit_research tool queried RAG + Metasploit

1:06:53 AM WARN WAF detected, throttling requests

1:06:53 AM INFO Reconnaissance phase started

1:06:53 AM INFO Assessment 'Corp Web App' created

Figure 5.4 — Écran de sortie du moteur : arbre d'agents, télémétrie (jetons, coût) et console live.

Strix Console
Security Operations

OPERATIONS

Dashboard

Assessments

Schedules

Reports

Logs

AI Assistant

ADMINISTRATION

Users

Roles

Audit Trail

Settings

N
Version: Strix

Schedules
Administrator · Admin

admin@strix.local

Schedules

Run assessments automatically on a recurring cron schedule.

+ New schedule

Name	Target	Cron	Next run	Last run	Status
Nightly external	app.example.com	0 2 * * *	—	—	paused

Figure 5.5 — Planification de tests récurrents (expression cron, prochaine exécution).

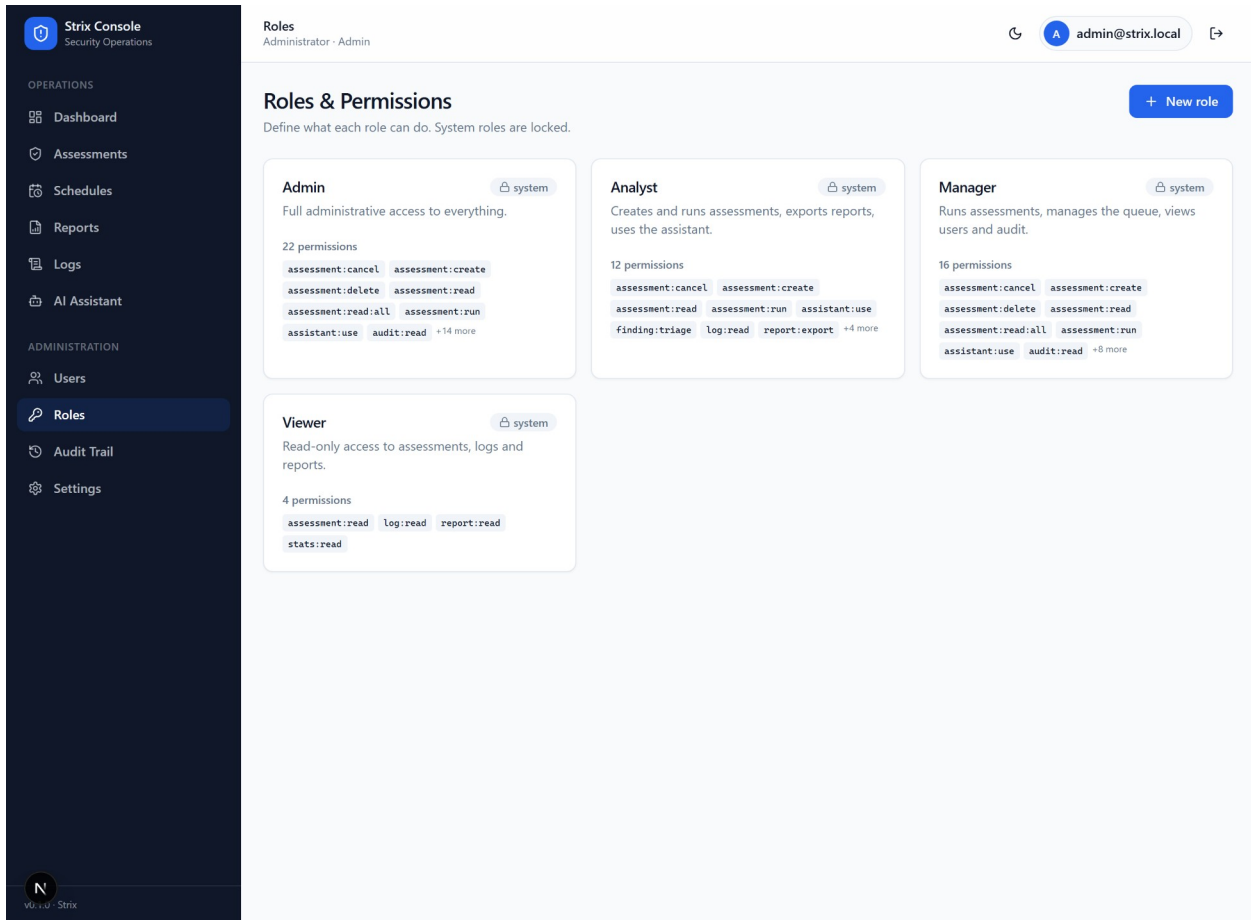


Figure 5.6 — Gestion des rôles et des permissions (RBAC).

5.4 Couverture de tests

La plateforme est couverte par un harnais de tests automatisés (pytest) exécuté en intégration continue, complété par des vérifications navigateur. L'ensemble des tests est au vert.

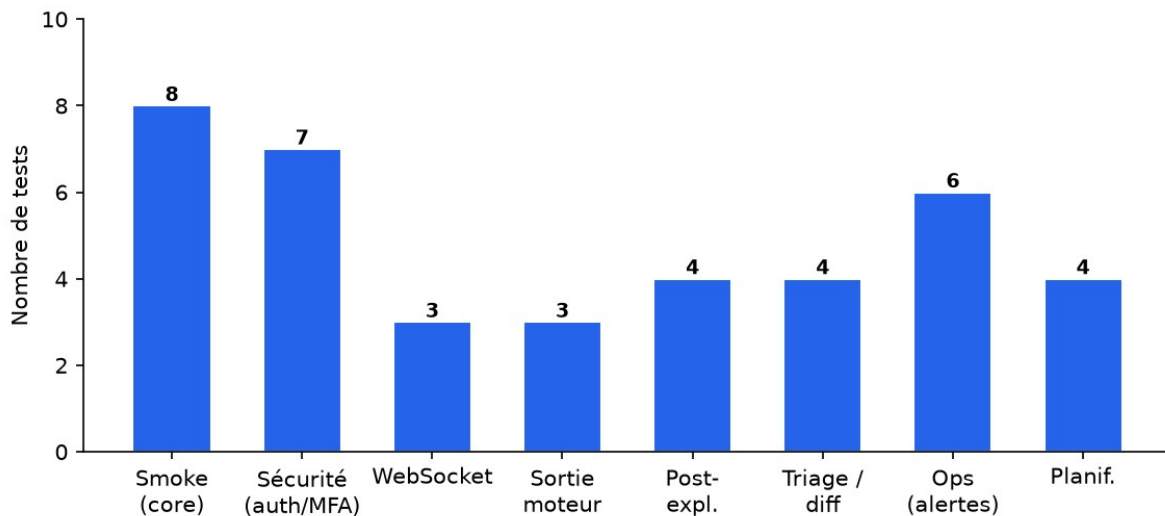


Figure 5.7 — Couverture de tests par module (39 tests, 100 % réussis).

5.5 Résultats et analyse comparative

À titre illustratif, la répartition des vulnérabilités par gravité sur une campagne de démonstration est présentée ci-dessous, suivie d'une analyse comparative qualitative entre un pentest manuel et l'usage de la plateforme.

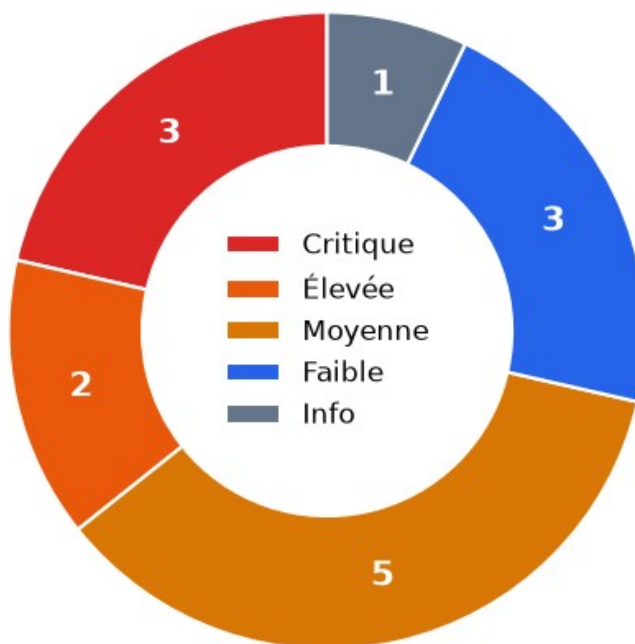


Figure 5.8 — Répartition des vulnérabilités par sévérité (campagne de démonstration).

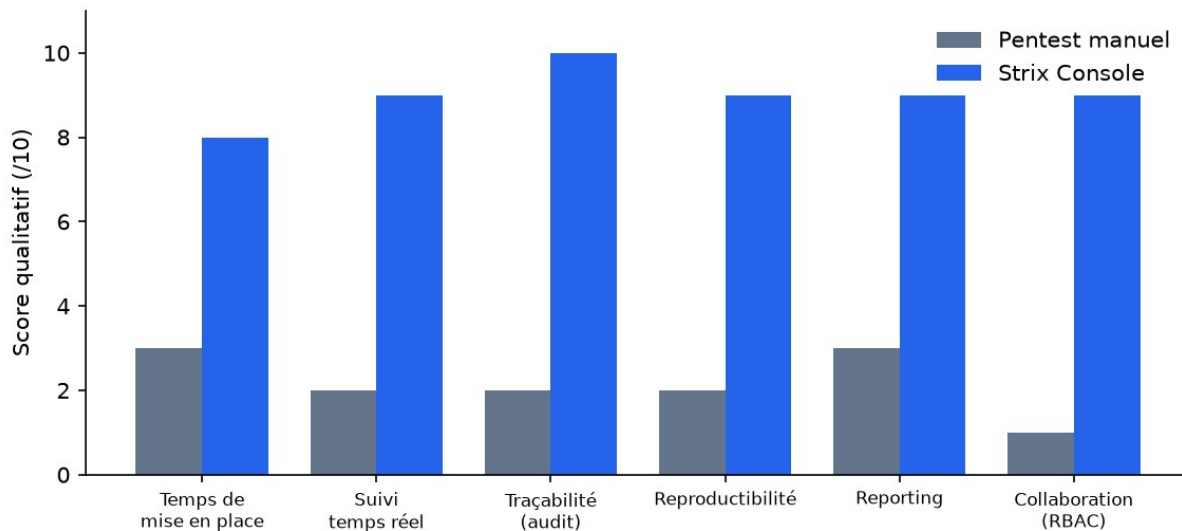


Figure 5.9 — Analyse comparative : approche manuelle vs plateforme.

Critère	Résultat
Tests automatisés	39 tests, 100 % réussis
Analyse statique (ruff)	Aucune erreur
Build frontend	14+ routes compilées, 0 vulnérabilité npm
Sécurité	Cookies httpOnly, CSRF, MFA, RBAC, audit chaîné vérifiés
Temps réel	WebSocket : état, journaux et vulnérabilités diffusés
Intégration moteur	exploit_research validé (RAG + module Metasploit)

Tableau 5 — Synthèse des résultats de validation.

5.6 Limites et perspectives

- **Ordonnanceur mono-processus** — APScheduler s'exécute en processus unique ; un déploiement multi-workers nécessiterait un processus d'ordonnancement dédié.
- **Runner en thread** — l'exécution des scans s'appuie sur un thread interne ; une extraction vers un worker RQ dédié améliorerait la durabilité.
- **Perspectives** — pièces jointes de preuve, scans multi-cibles, intégrations (Jira, SIEM), cartographie CVSS/CWE/OWASP, client TypeScript généré depuis l'OpenAPI.

5.7 Conclusion

L'évaluation confirme la robustesse fonctionnelle et la qualité de la plateforme, ainsi que la valeur de l'intégration Metasploit + RAG. Les limites identifiées ouvrent des pistes d'amélioration claires.

Conclusion générale

Ce projet de fin d'études a abordé l'automatisation avancée des tests d'intrusion par un système d'IA agentique fondé sur des LLM, en apportant deux contributions complémentaires. La première fiabilise la phase la plus délicate — l'exploitation — en intégrant Metasploit et un service RAG au moteur Strix via un outil déterministe unique, `exploit_research`, qui déplace la décision du modèle vers du code vérifiable et privilégie un exécuteur éprouvé.

La seconde transforme un outil en ligne de commande en une véritable console d'opérations de sécurité, Strix Console, dotée d'une authentification forte, d'un contrôle d'accès par rôles, d'un suivi temps réel, d'une visualisation de la sortie du moteur, d'une phase de post-exploitation, du triage des vulnérabilités, de la comparaison de scans, de la planification récurrente, du reporting et d'un journal d'audit inaltérable. L'ensemble, développé avec Next.js et FastAPI, a été validé par 39 tests automatisés et vérifié de bout en bout.

Au-delà des résultats obtenus, ce travail confirme la pertinence d'une approche hybride associant l'autonomie des agents IA à la robustesse d'outils éprouvés, encadrée par une plateforme sûre et gouvernée. Les perspectives — extraction du runner vers une file durable, intégrations métier (Jira, SIEM), cartographie normative des vulnérabilités et génération automatique du client d'API — dessinent une évolution naturelle vers un produit d'entreprise.

Bibliographie & Webographie

- [1] Strix — Moteur multi-agent open-source de tests d'intrusion. <https://github.com/usestrix/strix>
- [2] Rapid7 — Metasploit Framework. <https://www.metasploit.com>
- [3] OWASP — Web Security Testing Guide et OWASP Top 10. <https://owasp.org>
- [4] PentestGPT — LLM-driven penetration testing. Travaux de recherche associés.
- [5] FastAPI — Framework web asynchrone Python. <https://fastapi.tiangolo.com>
- [6] Next.js — Framework React (App Router). <https://nextjs.org>
- [7] SQLAlchemy 2.0 — ORM Python. <https://www.sqlalchemy.org>
- [8] APScheduler — Ordonnancement de tâches en Python. <https://apscheduler.readthedocs.io>
- [9] MITRE — Common Vulnerabilities and Exposures (CVE). <https://cve.mitre.org>
- [10] Retrieval-Augmented Generation (RAG) — Lewis et al., 2020.